

In response to Wayne's crossover project, I can claim that I agree with his conclusion in the third part that software engineering is a real type of engineering, and the consequences that can arise from unethically writing and developing software are as extreme as the consequences of unethical engineering in any other field. A key sentence that I read from the second part of Wayne's project was "I see this as a defense mechanism. If software is so different from trad, then it's okay for us to 'not be engineers'." Software engineering is typically not thought of as a "hardcore" engineering field. It is easy for developers to fall into the false "safety net" that their code is not directly interacting with users, unlike frontend we design, machinery like vehicles and robotics, and civil engineering products such as buildings. Therefore the impact of thinking about ethics in software development is not thought of enough. The truth is that software is probably the most impactful aspect of engineering when thinking about ethics. As Wayne says, software is unpredictable and continuously changing. Unlike concrete engineering products, software cannot have a single model and be multiplied thousands of times to create enough supply to meet user demand. Each piece of software needs to be tailored to each product it is being written for. Things like ensuring software is not unintentionally written from the viewpoint of one demographic only, making software unaware of some differences in people while also making it aware of other differences to ensure product usability, and ensuring that software cannot be utilized unethically by people who are not writing the software are all very important considerations to make, and often times it can be difficult to consider ethics if you are the only engineer working on a specific software product. Therefore, it is an obligation for software engineers to be trained to consider every ethical problem that can arise from their software.

After reading through Reamer's "Assessing Professionalism in Undergraduate Technical Communication Courses," I agree with his statements that in the current economic climate, the focus on companies looking to make as much profit as possible and hold high economic power has led to the unethical practice of withholding new knowledge and making subpar products in order to defeat competitors and make consumers pay more to maintain products even if they can be made it a better, long-lasting way. I also agree that under this definition of "professionalism," it is impossible for new software engineers to learn the true meaning of ethical consideration in their products. I like the models of professionalism that Reamer presented in the paper. Although hard skills are an important part of workplace competence, it is only one-third of the over professional requirements for a software engineer. Ethos is another third, where it is important for engineers to understand that their character and ethical obligations are just as important to consider as their technical skills. The last third, contribution to society, can also be considered an ethical obligation. In order to ensure software products will contribute positively to society, they must adhere to ethical standards to benefit users. In terms of ethos, I see the emphasis of character and ethics in my organization on campus, GEECS. Not only do we learn technical skills from upperclassmen and women in technology who work in research or industry, but we also are required to develop software to cater to different populations and their needs. We learn how to let community members set the standard for our software, and we only act as the technical expertise to make their ideas come to life. Learning how to separate personal thoughts and ideas from technical development is as important as incorporating original ideas to code. Ethics is not just about ensuring the code is impartial, but it is also about knowing when to incorporate your own ideas and when to hold back and ensure you are only considering the needs of the community you are working with. To summarize, ethics operates on a spectrum, where it is inevitable that some of your own morals will guide your software development. It is important to ensure code is usable by your entire user population, while also ensuring that you do not support projects that are designed to hinder one community or work against your basic moral compass. At the end of the day, if there is a project that you cannot develop without bias, it is best to not engage with it or ensure someone else who does not have bias takes the lead on it.